Università di Pisa

Corso di Laurea in Informatica Umanistica

Autorship Attribution su un Corpus di Tre Autori

Studente: Lavinia Rotellini
Matricola: 581553

Anno Accademico 2024/2025

1 Introduzione

Il progetto affronta la task di *autorship attribution*, utilizzando tre diversi modelli: un Support Vector Machine lineare e due reti neurali. Il Support Vector Machine è declinato in tre diverse applicazioni: è stato utilizzato inizialmente su testi rappresentati con features non lessicali estratte attraverso la piattaforma Profiling UD[1], poi su testi raffigurati utilizzando una rappresentazione basata su n-grammi di caratteri, lemmi e parts of speech, sempre estratti con Profiling UD, ed infine su una rappresentazione testuale che utilizza embeddings. Il report affronta le fasi di lavoro a partire dall'individuazione degli autori e la costruzione del dataset, discutendo poi dei risultati ottenuti con i Support Vector Machine ed infine l'applicazione e fine tuning delle reti neurali.

2 Il Dataset

Il primo step nella costruzione del dataset è stato la scelta degli autori oggetto di analisi, scelta basata sia sulla mole di produzione letteraria, così da assicurare la presenza di una quantità di paragrafi utile, sia sulla base del periodo storico, così da avere testi uniformi. Gli autori scelti appartengono al XIX secolo e sono George Eliot (Mary Ann Evans), Charles Dickens e Victor Hugo.
Per ognuno degli autori sono stati scelti romanzi diversi in modo da raggiungere un totale di 1200 paragrafi per il training set, 500 per il validation set e 698 per il test set. Per quanto riguarda George Eliot, i romanzi scelti sono i seguenti: *Middlemarch* per il training set, *The Mill on the Floss* per il validation set, *Adam Bede* e *Silas Marner* per il test set. Per Dickens, è stato scelto *David Copperfield* come training set, *Hard Times* per il validation set e *Great Expectations* per il test set. Infine, il training set di Hugo sono *Les Miserables*, il validation set *Toilers at the Sea* mentre il test set è *Notre Dame de Paris*. Se i testi sono stati presi in versione originale per Eliot e Dickens, i romanzi di Hugo sono stati recuperati in traduzione inglese. Aspetto interessante della ricerca mirava a comprendere se la traduzione dal francese mantenesse aspetti che la rendono maggiormente distinguibile rispetto agli altri testi, i risultati saranno presentati nelle sezioni successive.

2.1 Pulizia dei Testi

Si è resa necessaria una pulizia ad-hoc per ognuno dei testi, poiché presentavano peculiarità uniche. Per prima cosa, ogni testo è stato ripulito da informazioni altre rispetto al contenuto. Questo ha implicato la creazione di *regular expressions* specifiche per ogni romanzo, così da identificare esattamente i punti dove esso iniziava o terminava. Successivamente, il testo è stato diviso in paragrafi utilizzando tre diversi limitatori: la doppia riga vuota, due righe vuote separate con tabulazione o due righe vuote separate da spazio bianco. I paragrafi sono stati puliti eliminando gli spazi iniziali e finali e sostituendo a sequenze di spazi bianchi consecutivi uno spazio vuoto. Infine, sono stati mentenuti solo i paragrafi con una lunghezza tra i 50 ed i 100 caratteri.
Alcuni paragrafi presentavano poi delle caratteristiche non ottimali e sono stati sottoposti ad ulteriore preprocessing specifico:

- I paragrafi finali di *Notre Dame de Paris* erano interamente composti da un `digit` seguito dalla stringa `return`: questi paragrafi sono stati eliminati;
- In Great Expectations erano presenti paragrafi composti unicamente da sequenze di punti, anche essi sono stati eliminati.

I paragrafi sono stati salvati alla fine del preprocessing in liste separate di training, validation e test set per autore, poi utilizzate per la creazione del dizionario.

2.2 Creazione del Dizionario

Le analisi successive si sono basate sul dizionario `data.json`, strutturato come una collezione di dizionari secondari, ognuno dei quali corrisponde a un paragrafo. In ciascuno di questi, la chiave (`key`) è rappresentata da una stringa che identifica lo split, l'autore e un indice progressivo, ad esempio `train_Eliot_25`. Gli attributi invece, sono i seguenti:

- `text`: il paragrafo;

- `author`: l'autore del paragrafo;
- `split`: lo split di appartenenza del paragrafo.

Il dizionario è stato salvato con la libreria `json` per assicurare una facile fruizione nei i diversi notebook. Esso è inoltre stato utilizzato per il salvataggio di ogni istanza in un file `.txt` posizionato nella relativa cartella di train, validation o test, poi usato per l'etichettatura sulla piattaforma Profiling UD[1].

3 Classificatore Basato su Support Vector Machine lineari ed Informazioni Linguistiche non Lessicali

La prima task prevedeva l'allenamento di un classificatore basato su Support Vector Machine Lineari che utilizzasse come rappresentazione del testo unicamente informazioni linguistiche non lessicali. Le features sono state estratte attraverso la piattaforma *Profiling UD* e sono raggruppabili in sette insiemi principali che riguardano livelli di analisi diversi:

- Proprietà grezze del testo: ad esempio, la lunghezza in token e in frasi del testo;
- Varietà del lessico: stimata usando la *Type/Token Ration*;
- Informazione morfo-sintattica: che coinvolge aspetti come la distribuzione delle categorie grammaticali e la morfologia flessiva;
- Struttura del predicato verbale: come la distribuzione delle teste verbali;
- Strutture globali e locali dell'albero di parsing: come la sua lunghezza media o la lunghezza media degli archi di dipendenze;
- Relazioni sintattiche: per esempio, la distribuzione delle relazioni di dipendenza;
- Fenomeni di subordinazione: come la distribuzione di subordinate e frasi principali o la profondità delle subordinate inserite in un discorso.

In totale sono state utilizzate 126 features diverse, inserite in un nuovo attributo aggiunto per ogni subdizionario: il campo *features*. I vettori di features sono stati scalati utilizzando `MinMaxScaler`. Per effettuare la classificazione è stato scelto il modello `LinearSVC` dalla libreria `scikit-learn`. In una prima fase, è stata implementata una `GridSearch` con k=5, che ha portato alla selezione dei seguenti parametri:

C	class_weight	dual	fit_intercept	max_iter	penalty
1	balanced	False	False	50'000	l1

Il modello allenato sul training set è stato paragonato in ogni fold con un `DummyClassifier` implementato con strategia `uniform`: il classificatore dummy predice casualmente una classe tra quelle possibili, con probabilità uguali, dunque senza rispettare la distribuzione originale delle classi. In tabella 1 i risultati in accuracy per ogni fold:

Table 1: Risultati della cross-validation per fold

Fold	**Accuracy**	**DummyClassifier**
1	70%	33%
2	74%	33%
3	71%	34%
4	74%	33%
5	72%	31%

L'accuracy finale si stabilizza sul 72% per `LinearSVC`, mentre la baseline si ferma circa al 32%, come ci si attendeva poiché sono presenti tre classi da assegnare. L'analisi è proseguita con la predizione sui set di validation e test, che hanno riportato entrambi un 64% di accuracy versus il 32% del `DummyClassifier`.

Sono poi state esplorate le venti features più importanti sia per l'identificazione di ogni autore [1] che in generale per la risoluzione della task [2]. Si osserva che la feature più rilevante per l'identificazione di George Eliot è la distribuzione del tag `X`, comunemente impiegato per segnare elementi non riconoscibili e, pertanto, atipici rispetto alla norma. Tale evidenza risulta coerente con la significativa rilevanza attribuita anche alla feature `lexical density`, suggerendo l'ipotesi che Eliot adotti un uso peculiare del linguaggio. Inoltre, tratto condiviso con Dickens, notiamo che la distribuzione delle catene brevi di subordinate è fondamentale: questo rispecchia la tendenza dell'inglese a costruire periodi brevi. Per Hugo, infine, notiamo come la caratteristica più importante sia il numero di frasi, il che riflette la tipica lunghezza dei paragrafi da lui scritti, ma anche l'importanza della feature `upos_dist_SYM`, che fa riferimento alla presenza di simboli nel testo. Questo non è un fenomeno che ci si aspetterebbe all'interno di un romanzo, dove le parts of speech più frequenti dovrebbero essere nomi, aggettivi e verbi. Probabilmente, la folta presenza di questo token indica che il testo potrebbe presentare molti caratteri tipografici e decorativi, come la tipica †, ma anche problematiche di digitalizzazione e formattazione.

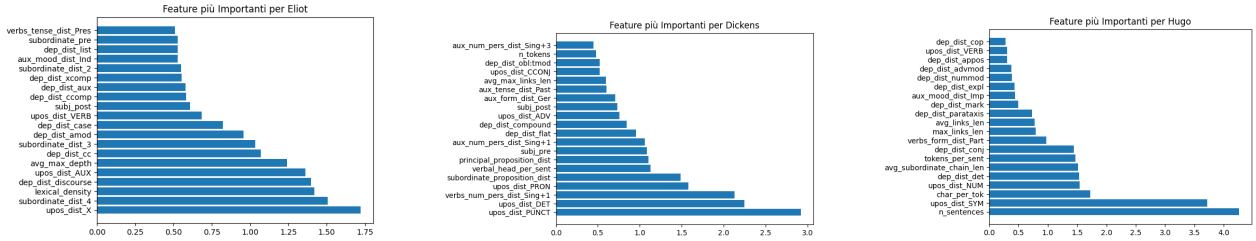

Figure 1: Features più importanti per l'identificazione dei tre autori

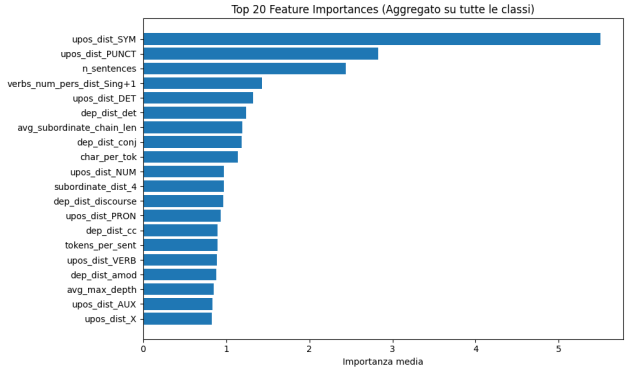

Figure 2: Le 20 feature più importanti.

Dall'immagine 2, si nota come due delle prime tre caratteristiche più rilevanti siano condivise con quelle di Hugo: sembra dunque che il modello trovi più semplice discriminare i suoi esempi. Il che era atteso, poiché Hugo è l'unico autore non inglese. Questa conclusione è confermata anche dal classification report del modello, dove i punteggi di precision, recall ed f1-score per Hugo si situano tutti attorno al 70%, mentre per Eliot e Dickens scendono tra il 52% ed il 64%.

4 Classificatore Basato su N-grammi

Il metodo successivamente impiegato per l'attribuzione d'autore consiste nell'applicazione di un classificatore Support Vector Machine lineare (`LinearSVC`), addestrato esclusivamente su una rappresentazione testuale basata sulla frequenza di token, lemmi, parti del discorso (POS) e caratteri. L'analisi è stata divisa in due parti: inizialmente si è proceduto testando le performance del modello su cinque configurazioni diverse per tipo di rappresentazione: il modello è stato dunque allenato e testato su ngrammi di parole, lemmi, POS e caratteri da uno a cinque. Successivamente, sulla base dei risultati ottenuti nella prima fase, sono state scelte le due configurazioni da ogni tipo di feature con il risultato più alto per creare le combinazioni finali. Il `LinearSVC` è stato addestrato sia nella prima che nella seconda fase utilizzando una Grid Search con `KFold` uguale a 5.
Nella prima fase di analisi, l'accuratezza sulla base delle singole features scelte è variata notevolmente, passando dal 36% dei 5-grams di lemmi al 75% dei bigrammi di parole. Al termine, le features selezionate sono state le seguenti:

- Unigrammi e bigrammi di words: rispettivamente con accuratezza 73% e 75%

- Unigrammi e bigrammi di lemmi: rispettivamente con accuratezza 72% e 73%
- Bigrammi e trigrammi di POS: rispettivamente con accuratezza 58% e 55%
- Trigrammi e 4-grams di caratteri: rispettivamente con accuratezza 70% e 74%

Le quali sono state poi combinate in cinque successivi casi di studio, ognuno paragonato con la baseline del `DummyClassifier` implementato nuovamente con strategia `uniform`:

Unigrammi di parole, bigrammi di parole, trigrammi di caratteri e 4-grams di caratteri La combinazione ha ottenuto un 84% di accuracy contro il 33% del `Dummy Classifier`

Unigrammi di parole, bigrammi di parole, unigrammi di lemmi, bigrammi di lemmi La combinazione ha ottenuto un 83% di accuracy contro il 34% del `Dummy Classifier`

Unigrammi di parole, bigrammi di parole, bigrammi di POS e trigrammi di POS La combinazione ha ottenuto un 84% di accuracy contro il 35% del `Dummy Classifier`

Unigrammi di parole, unigrammi di lemmi, trigrammi di POS, trigrammi di caratteri La combinazione ha ottenuto un 84% di accuracy contro il 33% del `Dummy Classifier`

Bigrammi di parole, bigrammi di lemmi, bigrammi di POS, 4-grams di caratteri La combinazione ha ottenuto un 85% di accuracy contro il 35% del `Dummy Classifier`

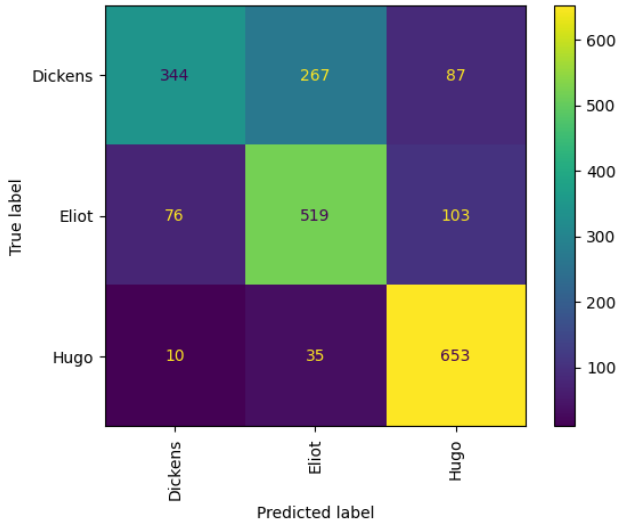

Figure 3: Confusion Matrix del modello finale: seconda task

Poiché i risultati in termini di accuratezza sono stabili attorno all'85% per ogni configurazione, si può affermare che l'aspetto più importante per ottenere prestazioni migliori sia l'aggiunta di informazione in generale, indipendentemente dal tipo purché si utilizzino caratteristiche utili. In ogni caso, si è proceduto con il testing utilizzando il modello risultante dal quinto caso: esso ha ottenuto un'accuracy sul test set del 72% e la seguente confusion matrix 3. In conclusione, il passaggio da una rappresentazione che utilizzava unicamente informazioni non lessicali ad una rappresentazione ad ngrammi ha portato un notevole incremento delle prestazioni: l'accuracy aumenta infatti di ben 8 punti, passando dal 64% al 72%.

5 Classificatore Basato su SVM Lineari e Word Embedding

In questa fase, la classificazione è stata implementata utilizzando i *word embeddings*, vettori multidimensionali densi che codificano il significato generale di una parola basandosi sui contesti in cui appare nei dati di addestramento. I word embeddings scelti sono stati gli `ukwac` a 128 dimensioni, disponibili sul sito di italianlp.it. Al fine di migliorare la qualità degli embeddings, è stato effettuato un preprocessing specifico ulteriore organizzato in tre step. Nel primo step, ogni numero è stato ritornato come stringa se compreso tra 0 e 2099, altrimenti ne veniva ritornata la lunghezza in formato stringa. In caso di fallimento della conversione, questo era sostituito con la stringa `@Dg`. Successivamente, si è proceduto con la normalizzazione delle parole: ognuna è stata identificata come URL se contenente la stringa *http*, altrimenti se presentava una lunghezza maggiore di 26 caratteri veniva sostituita con la stringa `__LONG-LONG__`. Infine, ciascuna parola è stata restituita conservando la maiuscola iniziale, qualora presente, oppure interamente in minuscolo, al fine di consentire al modello di preservare la distinzione tra le due forme.

Sono stati scelti tre metodi di aggregazione differenti per l'analisi: la media dei vettori delle parole utilizzate, la loro somma ed il loro massimo. La moltiplicazione è stata scartata per evitare l'utilizzo di vettori con numeri troppo piccoli che avrebbero potuto portare a perdita di informazione.
Le aggregazioni sono state analizzate basandosi sulle seguenti features:

- Il primo caso di studio ha analizzato l'aggregazione degli embeddings di tutte le parole all'interno dei testi;
- Il secondo caso di studio ha previsto l'aggregazione delle sole parole cariche di significato: nomi, verbi e aggettivi;
- Il terzo caso di studio si è basato sull'aggregazione di nomi e verbi, le parole più cariche di significato all'interno di un testo, insieme a parole povere di significato: avverbi, congiunzioni e interiezioni, per osservare se ci fossero utilizzi diversi di esse nei testi che potessero aiutare nella distizione;
- Inoltre, come baseline, si è utilizzato un `DummyClassifier` all'interno di ogni caso di studio, affiancato dalla classificazione effettuata utilizzando solo le parole povere di significato viste allo stadio precedente, così da osservare quanta informazione aggiungessero nomi e verbi.

Nella tabella sotto, i risultati in termini di accuracy sul set di validazione:

Table 2: Risultati dei diversi casi di studio

Caso di studio	**Media**	**Somma**	**Max**
Embeddings di tutte le parole	71%	71%	64%
Nomi, verbi, aggettivi	60%	60%	56%
Nomi, verbi, avverbi	59%	58%	55%
Nomi, verbi, congiunzioni	61%	59%	55%
Nomi, verbi, interiezioni	59%	59%	56%
Avverbi, congiunzioni, interiezioni	48%	50%	47%

Notiamo come il risultato migliore sia stato raggiunto dal modello che utilizza gli embeddings di tutte le parole nei testi, mentre le altre configurazioni si aggirano tutte attorno al 60% di accuracy grazie all'apporto di informazione portato da nomi e verbi. Non sono dunque rilevabili differenze particolari nell'utilizzo di avverbi, agettivi, congiunzioni ed interiezioni. La configurazione che utilizza avverbi, congiunzioni e interiezioni infine, ottiene punteggi significativamente più bassi per ogni metodo di aggregazione, confermando il presupposto che queste parole non contengono informazione decisiva per la classificazione.
Ogni caso di studio supera però la performance del `DummyClassifier` con strategia `uniform`, che come da configurazione non supera mai il 35%.
I modelli risultanti dalla media e somma di tutti gli embeddings sono passati poi alla fase di testing, ottenendo entrambi un 74% di accuracy, risultato che rappresenta un miglioramento rispetto alla rappresentazioni precedenti, per quanto il miglioramento rispetto alla rappresentazione ad ngrammi non sia decisivo.

6 Fine tuning di un Modello di Linguaggio Neurale

La fase finale di questa analisi ha previsto il fine tuning di due modelli neurali di linguaggio, `bert-base-cased` [2] e `distilbert-base-cased`[3]. Entrambi i modelli rappresentano varianti dell'architettura BERT e sono stati progettati per essere sottoposti a fine-tuning su compiti specifici. `bert-base-cased` utilizza un'architettura bidirezionale organizzata in 12 layer e 110 milioni di parametri, mentre `distilbert-base-cased` utilizza 6 layers e 66 milioni di parametri; entrambi i modelli mantengono la distinzione tra parole che iniziano con la minuscola e con la maiuscola, il che è fondamentale se vogliamo porre particolare attenzione ai nomi propri molto presenti all'interno dei romanzi.
Inizialmente lo studio è stato condotto utilizzando `distilbert`, modello più veloce e di dimensioni contenute che rappresenta un trade-off tra la complessità di BERT e prestazioni ottimali, ma si è poi deciso di procedere con l'applicazione anche di `bert-base-cased`, così da osservare se la differenza in dimensioni implicava una significativa differenza di accuratezza in questo task.

L'input di entrambi i modelli sono i paragrafi preprocessati ma grezzi, con le relative etichette che ne identificano l'autore. Il fine-tuning è stato eseguito per un totale di sei epoche: al termine, è stato salvato il

modello con validation loss migliore in entrambi i casi, così da mantenere quello con più alta capacità di generalizzazione. Gli iperparametri principali utilizzati sia per `bert-base-cased` che per `distilbert-base-cased` sono stati il `learning rate`, fissato a 2e-5, la `batch size` uguale ad 8 ed il `weight decay` uguale a 0.01.
La valutazione dei modelli è stata fatta considerando due diverse metriche principali: l'accuratezza, calcolata nativamente dal modello, e l'**f1-score**, per il quale è stata creata una funzione specifica.

Nelle figure 4 e 5 possiamo osservare le **loss curves** dei due modelli. Un aspetto particolarmente rilevante è la tendenza di entrambi i modelli a manifestare fenomeni di overfitting in una fase precoce del processo di addestramento: si nota come `bert-base-cased` entri in overfitting già alla seconda epoca, l'errore di training continua a diminuire, mentre l'errore sul validation aumenta velocemente. Per `distilbert-base-cased` il fenomeno è ancora più accentuato: già dalla prima epoca le due loss proseguono in direzioni diverse caratterizzando un adattamento sempre più preciso ai dati di training e conseguente perdita di capacità di generalizzazione. Tale comportamento risulta facilmente interpretabile alla luce delle ridotte dimensioni del dataset (solo 3600 esempi di training) e della complessità della task: i testi a disposizione sono estremamente brevi ed è necessario cogliere differenze di stile molto sottili per capirne l'autore, aspetto per il quale è probabilmente molto d'aiuto l'architettura più complessa di `bert-base-cased`.

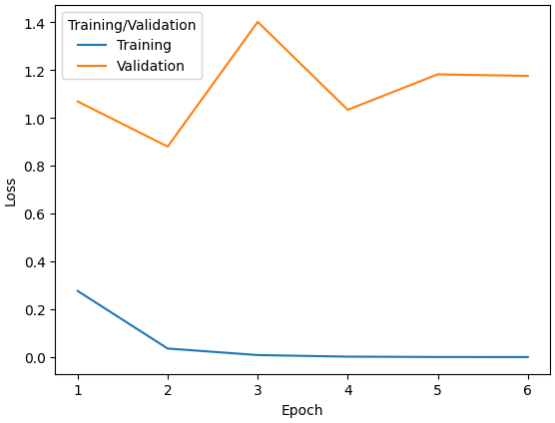

Figure 4: Loss Curve di BERT

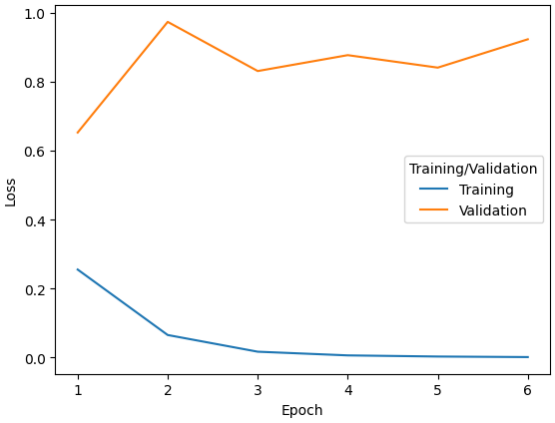

Figure 5: Loss Curve di DistillBERT

Nelle due tabelle 3 si possono osservare i risultati sul validation set epoca per epoca dei due modelli. Notiamo come, nonostante l'overfitting per `distilbert-base-cased` inizi fin dalla prima epoca, i valori di validation loss ed f1-score che ottiene sono decisamente migliori rispetto a quelli di `bert-base-cased`.

(a) Risultati di `bert-base-cased`

Epoca	Training Loss	Validation Loss	F1-Score
1	0.28	1.068	0.78
2	0.04	0.88	0.84
3	0.008	1.40	0.80
4	0.001	1.03	0.83
5	0.0004	1.18	0.84
6	0.0001	1.17	0.84

(b) Risultati di `distilbert-base-cased`

Epoca	Training Loss	Validation Loss	F1-Score
1	0.25	0.67	0.86
2	0.07	0.97	0.84
3	0.02	0.83	0.86
4	0.006	0.88	0.86
5	0.003	0.84	0.86
6	0.001	0.92	0.86

Table 3: Risultati delle fasi di training e validation

Eppure, questo non si traduce in una capacità di generalizzazione migliore: `bert-base-cased` infatti ha ottenuto un 86% di accuratezza, mentre `distilbert-base-cased` è arrivato ad un 80%, ottenendo comunque risultati migliori rispetto a tutti gli altri casi di studio. Sviluppi futuri del progetto potrebbero in ogni caso spingersi nella direzione di una ricerca di parametri ottimali e specifici per ognuno dei due modelli, così da mitigare l'overfitting e raggiungere performance più ottimali, approfondimenti che non sono stati eseguiti in questo studio a causa di limiti computazionali.
Nelle immagini sotto 7, le rispettive confusion matrices:

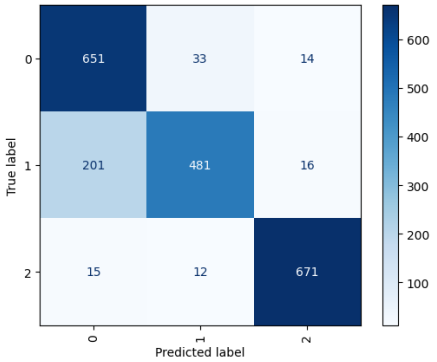

Figure 6: Confusion Matrix di BERT

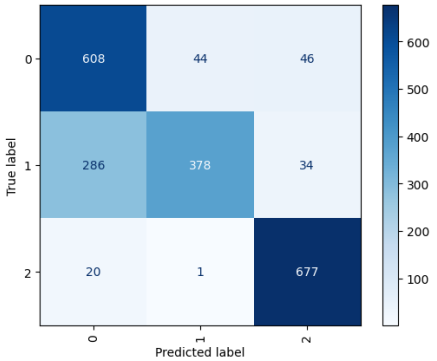

Figure 7: Confusion Matrix di DistilBERT

7 Conclusioni

In questa analisi sono stati testati diversi modelli e metodi per portare a termine un compito di authorship attribution su tre autori diversi: George Eliot (Mary Ann Evans), Charles Dickens e Victor Hugo. I risultati ottenuti riflettono la complessità dei metodi di rappresentazione e sviluppo utilizzati caso per caso: i modelli che hanno ottenuto performance peggiori sono quelli che considerano caratteristiche testuali più superficiali, come la rappresentazione con features non linguistiche o ad ngrammi, mentre l'utilizzo di rappresentazioni complesse che codificano anche il contesto come gli embeddings ed i modelli neurali del linguaggio sono stati in grado di cogliere sfumature di significato e sintattiche più profonde.
Aspetto interessante che potrebbe essere approfondito in futuro è la forte importanza del tag `upos_dist_SYM` per la classificazione dei testi appartenenti a Victor Hugo nella task 1: una maggiore comprensione di questo fenomeno potrebbe aiutare nel distinguere se si sono verificate problematiche di preprocessing, se il testo non è stato formattato correttamente o se semplicemente c'è una forte presenza di simboli, ipotesi che potrebbe applicarsi a testi dal forte contenuto religioso come quelli di Hugo. Disambiguare questa situazione permetterebbe di creare una pipeline ad hoc così da migliorare le prestazioni. In conclusione, questa analisi evidenzia sia le potenzialità dei moderni strumenti di linguistica computazionale per l'attribuzione autoriale, sia l'importanza di una selezione accurata delle rappresentazioni testuali. Studi futuri potranno estendere questa analisi a corpora più ampi o multilingue, esplorando ulteriormente l'interazione tra caratteristiche linguistiche e architetture computazionali.

References

[1] Dominique Brunato et al. "Profiling-ud: a tool for linguistic profiling of texts". In: *Proceedings of the Twelfth Language Resources and Evaluation Conference*. 2020, pp. 7145–7151.

[2] Jacob Devlin et al. "Bert: Pre-training of deep bidirectional transformers for language understanding". In: *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*. 2019, pp. 4171–4186.

[3] Victor Sanh et al. "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter". In: *arXiv preprint arXiv:1910.01108* (2019).